# THE DARK KNIGHT

## Engineering Documentation

WRO Future Engineers 2026

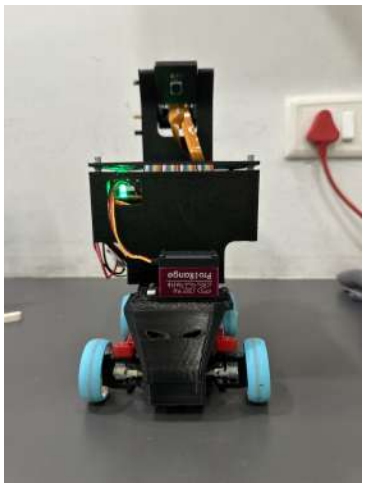

Figure 1: The Dark Knight — final robot.

| | |
|---|---|
| **Team:** | Current |
| **Team members:** | Darsh Makadia, Ehan Mansuri |
| **Coach:** | Sunil Solanki |
| **Competition level:** | Nationals |
| **Category:** | WRO Future Engineers |

**Engineering philosophy**

Find the best approach with the materials available and give the best possible output.

Repository: https://github.com/darsh-makadia/Team-Current-WRO-FE-2026

# Contents

**How to read this document.** Blue blocks indicate process steps, gold diamonds indicate decisions, green blocks indicate resulting actions or end states, and dashed arrows indicate feedback or repeated control loops.

# 1 Executive Summary

## 1.1 What we built

The Dark Knight is our autonomous WRO Future Engineers robot, developed by Team Current for the 2026 season. The robot combines servo steering, four-wheel drive, a mechanical differential, custom CAD/3D-printed structures and LEGO mechanical components.

The project was developed around one central idea:

> **The best solution is not necessarily the most complicated solution; it is the solution that gives the best overall result under the real constraints of the robot, the rules and the available materials.**

The design therefore deliberately combines technologies instead of forcing the entire robot into a single construction method.

## 1.2 Competition objectives

The intended autonomous system had to address the major Future Engineers tasks:

- complete the Open Challenge laps,
- operate in both possible track directions,
- detect and respond to track markers,
- follow the black track boundaries,
- detect coloured obstacles,
- navigate around obstacles,
- enter and align within the parking area,
- use the available sensing information to maintain reliable orientation.

# 2 Requirements, Constraints and Design Targets

## 2.1 Design requirements

Before optimisation, the robot had to satisfy several practical requirements simultaneously.

Table 1: Primary design requirements

| Requirement | Engineering interpretation |
|---|---|
| Autonomous operation | The robot must perceive the track and generate steering decisions without manual driving. |
| Track following | The system must extract useful information from the black track boundaries. |
| Direction awareness | The system must distinguish the two marker directions and count the appropriate marker colour. |
| Obstacle perception | The camera system must identify relevant coloured obstacles. |
| Parking perception | The robot needs useful rearward visual information for the parking task. |
| Mechanical compliance | The drivetrain must use a mechanically appropriate differential solution. |
| Electrical reliability | The Pi, cameras, motor driver and motors must receive appropriate power. |
| Repeatability | The robot should behave consistently across repeated runs rather than succeed only once. |

### 2.2 Major constraints

The most important practical constraints were:

- limited battery availability;
- only one battery could be charged at a time;
- Raspberry Pi power was not sufficient to directly supply the motor system;
- motor-driver current capability created a risk when motors stalled;
- camera height and angle directly affected what could be detected;
- the drivetrain had to satisfy the applicable differential constraint;
- the robot had to combine LEGO components with custom structures;
- development time limited how much simultaneous challenge testing could be performed.

### 2.3 Design philosophy

Rather than treating each constraint separately, the team treated the robot as a coupled system.

For example:

$$\text{Motor choice} \rightarrow \text{current risk} \rightarrow \text{power architecture} \rightarrow \text{available speed} \rightarrow \text{steering behaviour}.$$

Similarly:

$$\text{Camera height} \rightarrow \text{field of view} \rightarrow \text{obstacle detection} \rightarrow \text{steering target} \rightarrow \text{collision risk}.$$

This systems-level interaction became one of the main lessons of the project.

## 3 Development Evolution

### 3.1 Overview

The robot passed through several major design stages. The exact intermediate configurations were not all retained as separate CAD assemblies, but the physical development process can be summarised as:

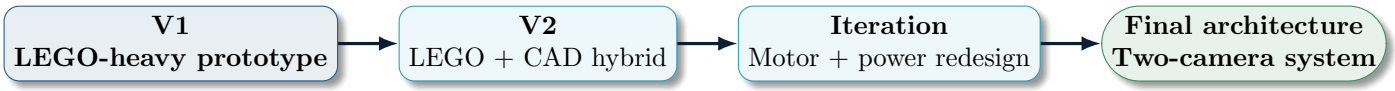


Figure 2: Major development stages of The Dark Knight.

## 3.2 Version 1: LEGO structure

The first version was built almost entirely from LEGO components. It used a LEGO Medium Motor and a LEGO-oriented mechanical differential.

**Stage clarification:** The V1 and V2 photographs below document the mechanical-development stages, not the final perception system. Neither prototype photograph shows the cameras because the camera architecture was introduced and refined later. The two-camera system belongs to the later development stage and should therefore not be expected in the original V1/V2 photographs.

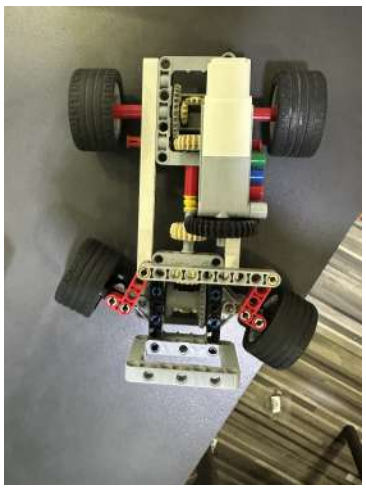

Figure 3: Version 1: the original LEGO-heavy prototype.

The LEGO prototype was useful because it enabled fast physical experimentation. Gear relationships, axle positions and the differential could be modified without manufacturing custom parts.

However, the structure also created limitations. The frame became relatively heavy and it was difficult to produce the rigid, purpose-specific mounting geometry required for cameras and electronics. The LEGO EV3 Medium Motor used in V1 had a maximum speed of 240 to 250 RPM, which was quite slow for the operation of the robot, which caused the need of a new motor as well.

## 3.3 Version 2: CAD structure with LEGO drivetrain

The next design retained important LEGO mechanical components but introduced CAD-designed structural components.

Figure 4: Intermediate LEGO/CAD hybrid design.

The structural base and circuit base were redesigned using CAD. The differential and motor arrangement remained LEGO-oriented.

This was not simply a replacement of LEGO by CAD. It was the first explicit use of a hybrid design philosophy.

### 3.4 Final architecture

The final robot uses CAD where rigidity and custom geometry are important, while retaining LEGO where small mechanical precision and modularity are advantageous.

The final custom parts include:

- chassis,
- 36 teeth gear
- camera mount,
- camera case,
- circuit box,
- circuit-box lid.
- base for chassis

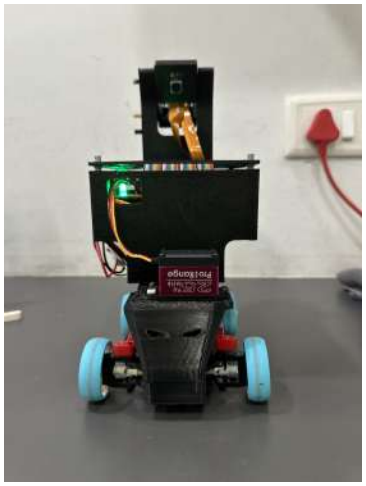

Figure 5: Final robot — three-quarter view.

### 3.5 Development timeline

Table 2: Development decisions and their triggers

| Stage | Main problem/requirement | Resulting change |
|---|---|---|
| V1 | Rapid mechanical prototyping | LEGO-heavy robot |
| V2 | Need for stronger/custom structure | CAD chassis introduced |
| Drivetrain iteration | Combating Large Turns | Mechanical differential retained |
| Motor iteration | Stalling/current risk | Johnson motor replaced by JGB37 |
| Power iteration | Pi could not supply motor system | Separate power branches |
| Camera iteration | Camera placement affected perception | Adjustable CAD camera mount |
| Vision iteration | Multiple visual tasks | Two-camera architecture |
| Control iteration | Duplicate marker counting | Rising-edge + cooldown |
| Parking iteration | Timed turns lacked reliable orientation | IMU heading feedback |

# 4 Mechanical Architecture

## 4.1 Final mechanical layout

The assembled robot was re-measured after the latest hardware changes. The confirmed overall dimensions are 24 cm in length, 13 cm in width and 27.5 cm in height.

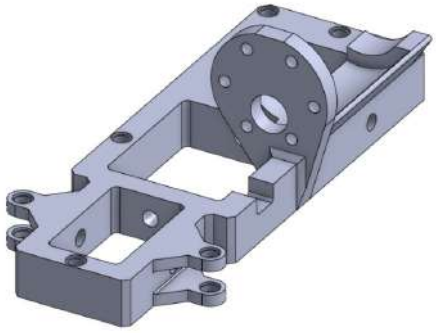

Figure 6: Final CAD chassis.

The final mechanical system is organised around a rigid custom chassis with dedicated mounting locations for the motors, steering servo, electronics and cameras.

The final measured specifications are:

Table 3: Final robot specifications

| Parameter | Specification |
|---|---|
| Length | 24 cm |
| Width | 13 cm |
| Height | 27.5 cm |
| Mass | 863 g |
| Wheel diameter | 43.2 mm |
| Gear ratio | 1.8:1 |
| Drive configuration | Four-wheel drive |
| Drive motor | JGB37-520 DC 12 V 600 RPM |
| Steering | Servo motor |

## 4.2 Four-wheel drive decision

During the final parallel parking maneuver, the robot needs to make a turn with a large steering angle. If we use only two-wheel drive, the powered wheels will push the robot while the steered wheels are not powered. Because the steered wheels are already at a large angle, this can cause wheel slip and drifting, making the robot difficult to control accurately.

To solve this, we decided to use four-wheel drive. With all four wheels powered, the steered wheels also generate driving force in the direction in which they are turned. This helps the robot follow the turning direction instead of pushing sideways, reducing drift and improving traction and control during sharp turns.

Therefore, four-wheel drive gives us better control, less drifting, and more precise movement during the final parallel parking maneuver.

## 4.3 Mechanical differential: the design problem

The differential became one of the most important mechanical design problems.

The team wanted a four-wheel-drive architecture while remaining within the applicable rule constraints. An electrical differential could not be used, so the required difference in wheel rotational behaviour had to be achieved mechanically.

The solution was a mechanical differential.

Power from the drive motor is transmitted through a central driveshaft running along the chassis centreline. This drives the gear mechanisms at both the front and rear axles, where bevel gears transfer the rotation into mechanical differentials. The two differentials then distribute power independently to the left and right wheels, providing four-wheel drive while allowing the wheels on each axle to rotate at different speeds during turns.

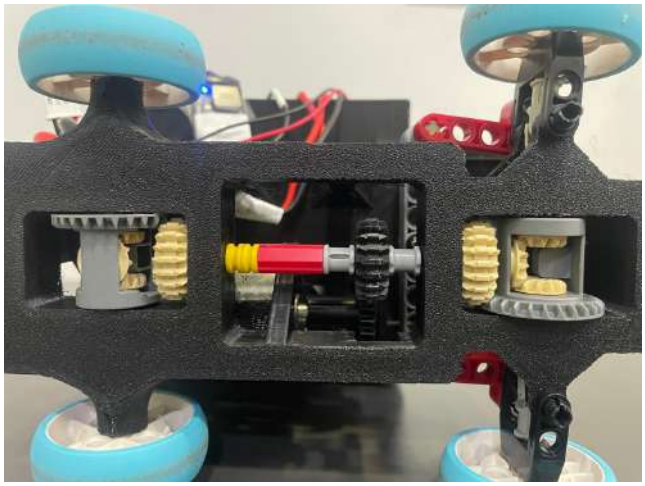

Figure 7: Mechanical differential in the final drivetrain.

The LEGO differential was retained because the available LEGO gear components made it possible to create a compact mechanical solution without designing and manufacturing a custom gearset.

| | |
|---|---|
| **Problem** | The desired drive system required all four wheels to be powered while the differential implementation had to remain mechanical. |
| **Constraint** | An electrical differential was not an acceptable solution under the applicable constraint. |
| **Decision** | A LEGO-oriented mechanical differential was integrated into the four-wheel-drive system. |
| **Result / next step** | The final drivetrain preserved the desired four-wheel-drive concept while avoiding an electrical differential. |

## 4.4 Why not make the entire drivetrain CAD?

A custom CAD differential would have introduced several new engineering problems:

- gear tooth geometry,
- backlash,
- axle alignment,
- tolerances,
- friction,
- manufacturing accuracy.

LEGO already provided a mechanically precise, modular differential. Therefore, replacing it would have added manufacturing complexity without providing an obvious performance advantage.

This was a recurring principle:

> **Do not redesign a component merely because a newer technology is available. Redesign it when the new technology solves a real problem.**

## 4.5 Steering architecture

The robot uses a servo motor for steering.

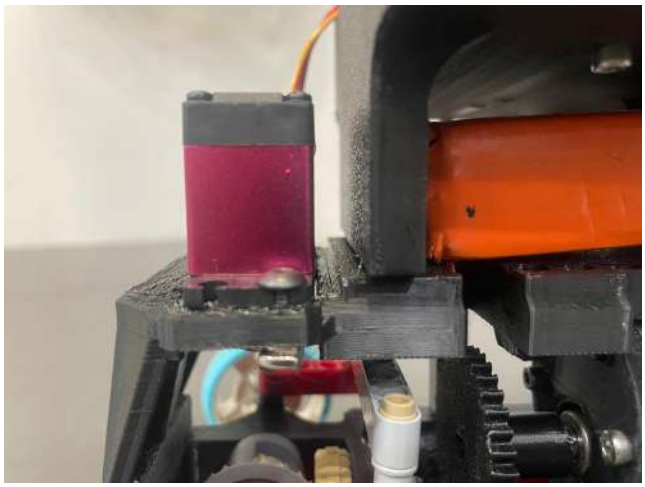

Figure 8: Final steering mechanism and dedicated servo mounting geometry.

The steering servo is secured with dedicated mounting holes and screws. The final system does not exhibit significant steering backlash in the team's testing, and the steering response can be adjusted through the software's calibrated centre and limits.

The Open Challenge configuration uses:

$$\theta_{\text{center}} = 75^\circ, \qquad \theta_{\text{min}} = 35^\circ, \qquad \theta_{\text{max}} = 115^\circ.$$

These values were experimentally tuned rather than derived solely from the servo datasheet.

## 4.6 Why Ackermann steering was not adopted

Ackermann steering was considered.

The main reason for not adopting it was that it would add mechanical complexity and interact with the existing robot structure. The team instead retained a simpler servo-steering architecture that was easier to integrate and tune.

This was a deliberate simplicity/performance trade-off.

## 4.7 Chassis rigidity and weight distribution

The final chassis is dense and rigid enough for the current operating conditions. The mass distribution is approximately centred.

However, the team observed that at very high speed the robot could become dynamically unstable and could even lift the front wheels.(Note: This happened when we were using the Johnson 1000 RPM Motor)

This observation influenced the decision not to optimise solely for maximum speed.

Figure 9: Top view showing the overall layout and component distribution.

### 4.8 Rules Compliance

Table 4: Compliance against WRO Future Engineers 2026 vehicle rules

| Rule | Requirement | The Dark Knight |
|---|---|---|
| 9.17 / 11.1 | Footprint $\leq$ 300 $\times$ 200 mm, height $\leq$ 300 mm | 240 $\times$ 130 $\times$ 275 mm — within limits |
| 11.2 | Weight $\leq$ 1.5 kg | 863 g — within limits |
| 11.3 | Four wheels, one steering actuator, must be FWD/RWD/4WD (not a differential-wheeled/skid-steer base) | Four wheels, one servo, 4WD via a single mechanical differential and driveshaft |
| 11.4 | No omnidirectional, ball, or spherical wheels | Standard wheels only |
| 11.5 / 11.13 | Max. two driving motors; motors must not be connected independently to each side (no electronic differential) | One drive motor, connected through gearing and a single mechanical differential to all four wheels |
| 9.10 / 9.11 | One switch to power on; one start button | One power switch; one push button (GPIO18) |
| 11.10 | No active wireless communication during a round | Wi-Fi/Bluetooth disabled during runs |

## 5 LEGO and CAD Integration

### 5.1 Design philosophy

The project found that LEGO and CAD were complementary rather than competing technologies.

Table 5: Where each manufacturing method was strongest

| Design requirement | LEGO | CAD / 3D printing |
|---|---|---|
| Differential | Strong | Requires custom gear design |
| Small gears | Strong | Manufacturing/tolerance challenge |
| Axles/connectors | Strong | Unnecessary to redesign |
| Rigid chassis | Limited | Strong |
| Camera mount | Initially unstable | Strong and adjustable |
| Electronics box | Limited | Strong |
| Custom mounting holes | Limited | Strong |
| Rapid mechanical experimentation | Strong | Slower |

## 5.2 Camera mount redesign

The first camera mount was made with LEGO.

The problem was not that LEGO was mechanically weak in general. The specific problem was that the camera mount was too wobbly for a sensor whose output directly affected steering and obstacle detection.

The mount was therefore redesigned using CAD.

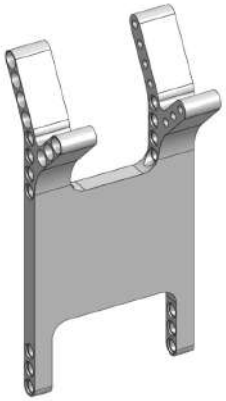

Figure 10: CAD camera mount.

The new mount allowed the camera position and angle to be changed more deliberately.

## 5.3 Printed components

The custom parts were manufactured using PLA on an Anycubic Cobra 2 Neo.

The main printed components were:

- chassis,
- camera mount,
- camera case,
- circuit/electronics box,
- circuit-box lid.
- base on the chassis
- 36 teeth gear

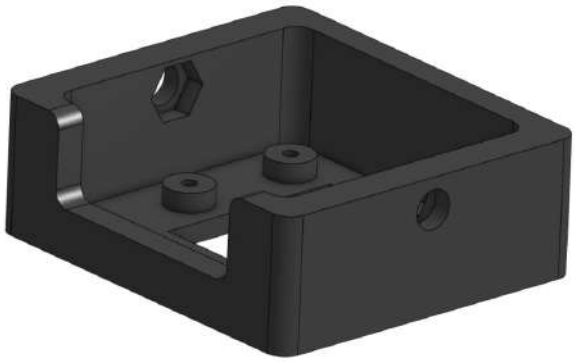

Figure 11: CAD camera case.

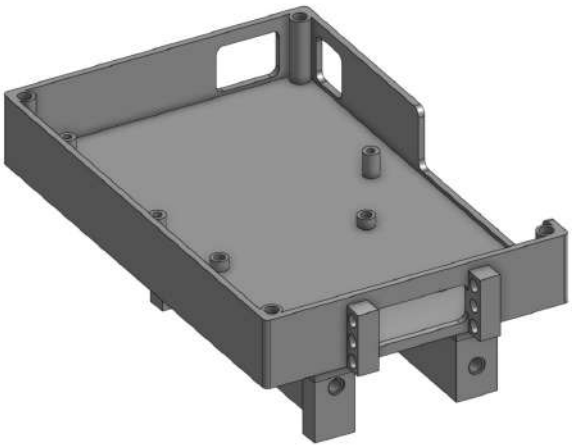

Figure 12: CAD-designed electronics enclosure.

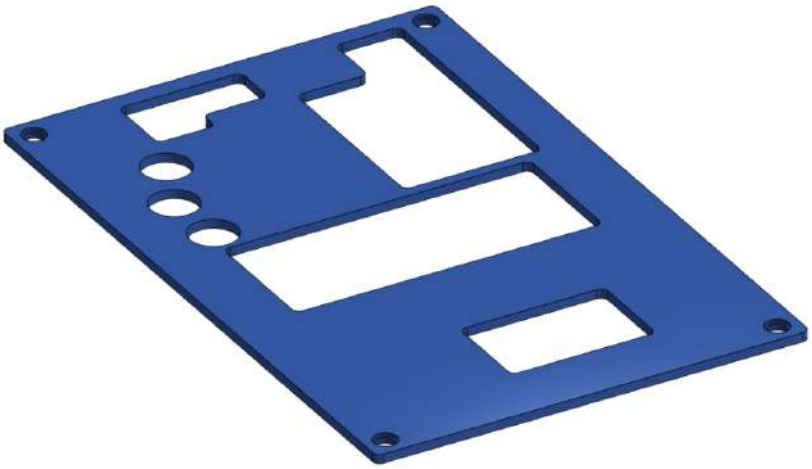

Figure 13: CAD-designed circuit-box lid.

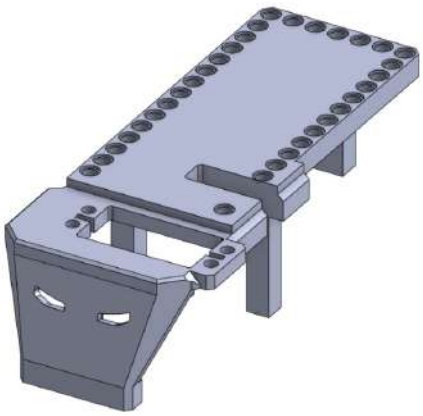

Figure 14: Base on the chassis.

# 6 Motor Engineering and Drive Performance

## 6.1 Initial motor: Johnson 1000 RPM

The initial motor choice prioritised high rotational speed.

During development, motor stalling became a significant concern.

A stalled DC motor can demand substantially more current than it does under normal rotation. If the driver cannot safely provide that current, the driver can overheat or become damaged.

The team therefore treated stalling not simply as a loss of movement but as an electrical-system risk.

## 6.2 Motor replacement

The final motor was changed to:

**JGB37-520 DC 12 V 600 RPM**

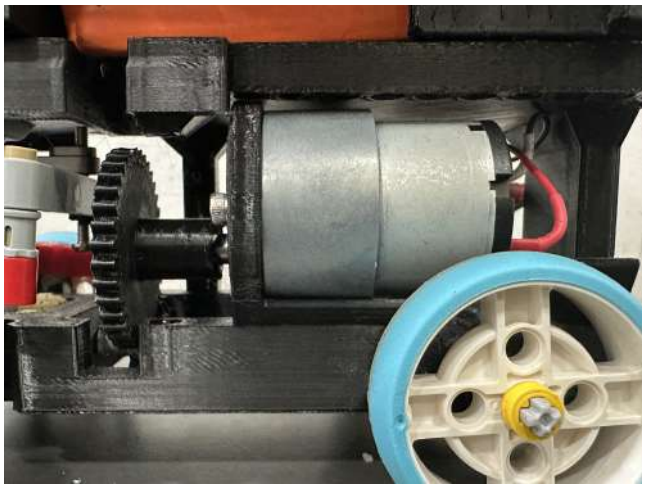

Figure 15: Final JGB37-520 motor.

The final motor also provided a higher-RPM operating point than the earlier LEGO motor arrangement while avoiding the same level of concern associated with the Johnson motor.

## 6.3 Final motor specifications

Our final drive motor is a JGB37-520 geared DC motor. The specifications we recorded from the motor's official product information are:

Table 6: Final drive motor specifications

| Parameter | Value |
|---|---|
| Rated voltage | 12 V DC |
| No-load speed | 600 RPM |
| Rated torque | 0.15 kg-cm |
| Stall torque | 0.60 kg-cm |
| No-load current | approximately 100 mA |
| Rated current | approximately 190–200 mA |
| Stall current | 1.2 A |
| Gearbox ratio | 10:1 |

We use 0.15 kg-cm as the rated torque and 0.60 kg-cm as the stall torque in this documentation. These are the minimum values from the motor specification we are using.

## 6.4 Drive gearing

The final drivetrain uses one drive motor to transmit power to all four wheels through the mechanical drivetrain and differential. The external gear pair consists of a custom 3D-printed 36-tooth gear meshing with a LEGO 20-tooth gear, giving a tooth-count ratio of

$$\frac{36}{20} = 1.8 : 1.$$

The 36-tooth gear is the custom component in the drivetrain. The selected gear relationship was retained because it increases output speed at the wheel, at the cost of available torque, as detailed below. For a meshing gear pair, the torque and speed trade-off depends on which gear is the driving member: a reduction in rotational speed produces a corresponding increase in available torque, while a speed increase produces

a corresponding reduction in torque. The 1.8:1 relationship is therefore documented as a mechanical trade-off rather than as a claim that gearing creates additional power.

Applying this 1.8:1 ratio to the motor's rated specifications gives a final stall torque of

$$\frac{0.60}{1.8} \approx 0.333 \text{ kg-cm}$$

and a final rated torque of

$$\frac{0.15}{1.8} \approx 0.0833 \text{ kg-cm}.$$

Going from the 36-tooth driving gear to the 20-tooth driven gear therefore reduces available torque in exchange for increased output speed. This was the intended outcome: for the Open and Obstacle Challenges, the team judged that additional speed mattered more than additional torque, since the robot's mass is low enough that the reduced torque is still sufficient to drive the wheels reliably.

At the final rated torque of 0.0833 kg-cm, the thrust force at the 21.6 mm wheel radius is approximately 0.038 N, or roughly 3.8 grams-force per driven wheel. Summed across four driven wheels this is small relative to the robot's 863 g weight, but is judged sufficient because the robot's low overall mass and the differential's ability to distribute torque to whichever wheels have traction keep the required force for steady-state motion low.

## 6.5 36-tooth custom gear

The 36-tooth gear was designed and 3D printed specifically for the final drivetrain. The CAD render is included as a focused visual reference so that the custom drivetrain component is easier to identify within the broader mechanical system.

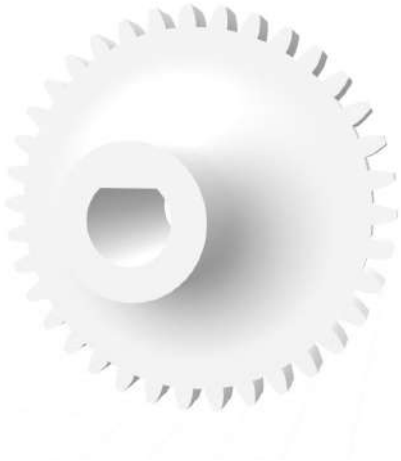

Figure 16: Custom 36-tooth gear used in the final drivetrain.

## 6.6 Why maximum RPM was not the objective

At first glance, higher motor RPM appears beneficial because it can increase speed. In an autonomous robot, however, the useful objective is not simply:

$$\max(\text{speed}).$$

The more relevant objective is closer to:

$$\max(\text{successful completion rate}).$$

A faster robot that:

- overshoots steering corrections,
- detects obstacles too late,
- destabilises its chassis,
- or damages a motor driver

can perform worse than a slightly slower but more controllable robot.

The team's later testing therefore favoured accuracy and repeatability over maximum theoretical speed.

### 6.7 Speed and stability observation

An important physical observation was that making the robot travel very fast could cause a "wheelie", where the front wheels lifted. This was observed while the robot was still using the higher-power Johnson 1000 RPM motor, before it was replaced with the lower-torque JGB37-520. The observation contributed to the decision not to pursue maximum available torque in the final motor choice.

## 7 Electrical and Power Architecture

### 7.1 Initial problem

The Raspberry Pi was not an appropriate power source for the motor drivers and motors.

The team therefore reorganised the system into branches so that high-power motor loads would not depend on the Pi's supply path.

### 7.2 Final power concept

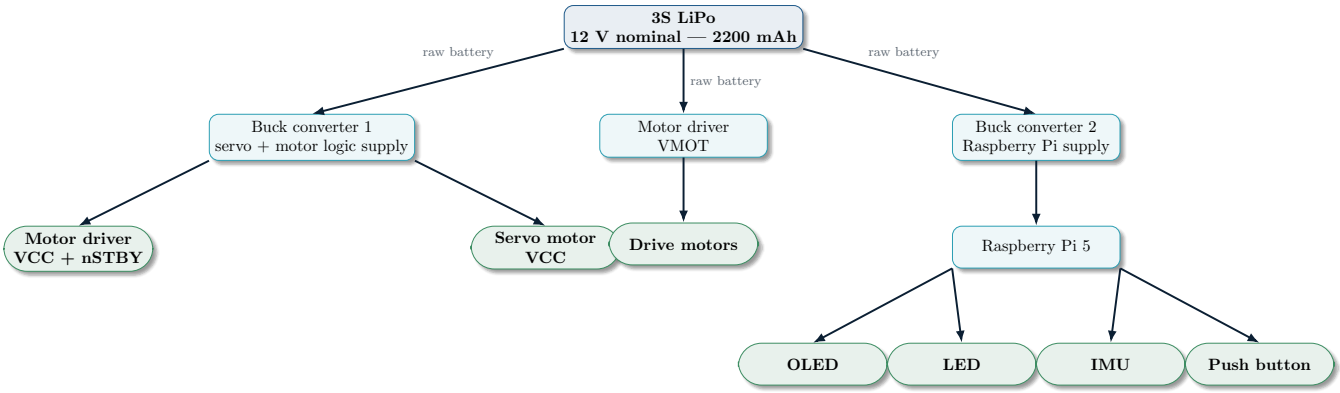


Figure 17: Final power architecture. The raw 3S LiPo battery supplies three main branches: Buck Converter 1 for the motor driver's VCC and nSTBY connections and the servo motor VCC; the motor driver's VMOT input for the drive motors; and Buck Converter 2 for the Raspberry Pi 5 and its connected peripherals.

Figure 18: Electronics enclosure.

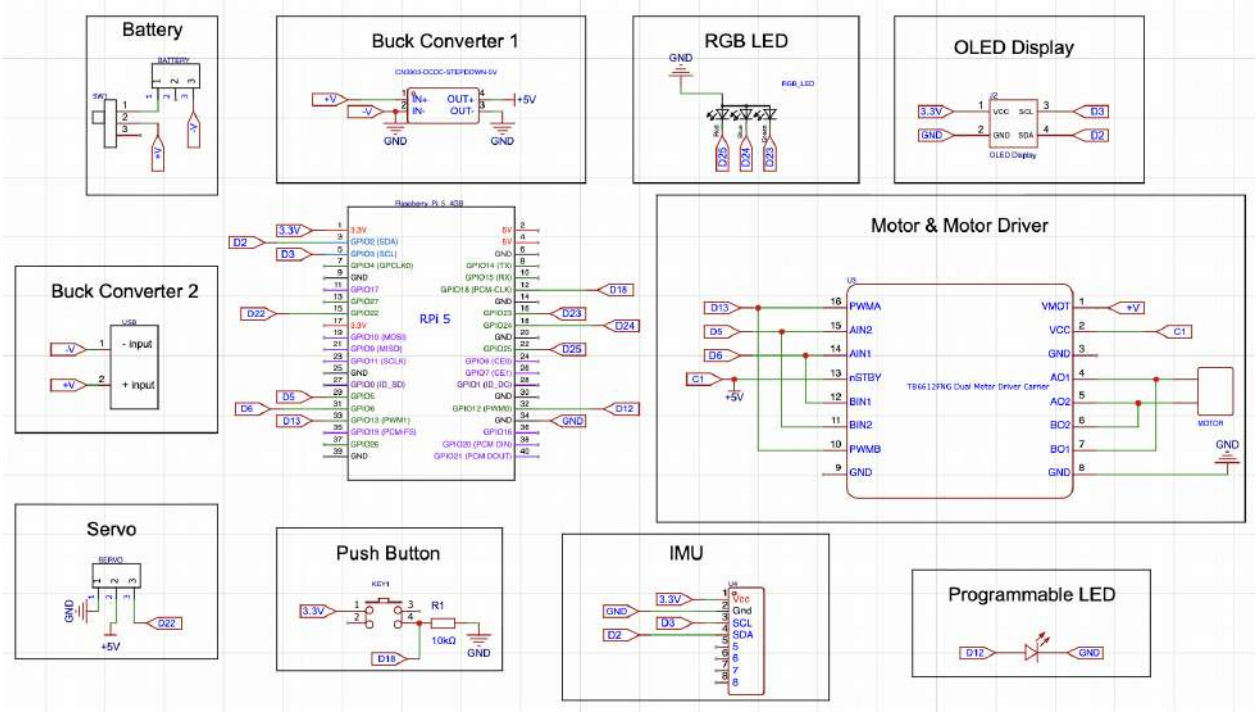


Figure 19: Final EasyEDA schematic showing the Raspberry Pi, motor driver, power branches, I2C devices, servo, LEDs and push button.

Figure 20: Closed CAD/printed electronics enclosure on the robot.

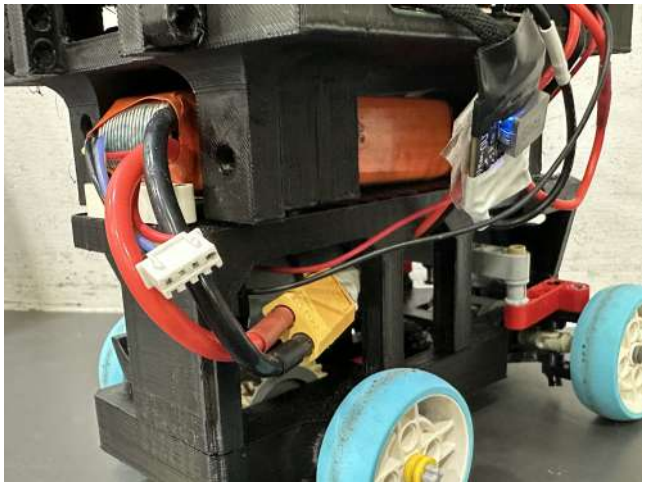

Figure 21: Battery installation and power source.

## 7.3 Power measurements

Table 7: Measured voltage values

| Point | Condition | Value |
|---|---|---|
| LiPo battery | Before run | 11.1 V |
| LiPo battery | After multiple runs | 10.8 V |
| Buck 1 output | Motors OFF | 5.0 V |
| Buck 1 output | Robot moving | 5.0 V |
| Pi supply | Idle | 5.0 V |
| Pi supply | Moving | 5.0 V |
| Motor driver | Motors OFF | 11.1 V |
| Motor driver | Motors ON | 10.8 V |

Operation is stopped if the loaded battery voltage falls below approximately 10.5 V. In practice, battery voltage was not a limiting factor during development: with three batteries available, a pack was swapped for a fresh one once performance began to degrade, rather than discharging any single pack toward its cutoff.

## 7.4 What the measurements tell us

These measurements were taken while the robot was operating from its battery during testing. The measured supply voltages remained approximately stable at the measured points under those tested conditions.

This does not prove that there are no transient voltage drops or current spikes. It only establishes that the multimeter measurements taken during testing showed the stated values.

This distinction is important because a multimeter is not sufficient to characterise every fast electrical transient.

## 7.5 Low-voltage warning

Earlier in development the Pi produced a low-voltage warning.

At the latest recorded state, the low-voltage warning was not occurring.

The original warning contributed to the decision to improve the power distribution rather than assuming that the Pi could safely supply the entire system.

## 7.6 Schematic-level connections

The enlarged electrical schematic is the primary record of the implemented connections. The separate pin-to-component and component-to-component tables were removed because they repeated information already visible in the schematic without improving readability.

The motor-driver channels are wired in parallel for the single drive motor. The duplicated motor-driver connections are intentional: both channels are used together to share the motor current rather than to control two separate motors.

## 7.7 Electrical power distribution

The final schematic separates the raw battery branch from the regulated logic branch. The battery rail supplies the motor-driver VMOT and the buck-converter inputs. Buck 1 produces the regulated +5 V rail used by the servo and logic-side components shown in the schematic. Buck 2 is rated at 5 V and 5 A and forms a separate regulated branch. The Raspberry Pi provides the regulated 3.3 V rail for the OLED and IMU. All branches share a common ground.

Table 8: Electrical power distribution.

| Rail / net | Voltage level | Power source | Target modules / connections |
|---|---|---|---|
| **+V / -V** | Raw battery voltage | 3S LiPo via switch | Buck 1 input, Buck 2 input, motor-driver VMOT |
| **+5 V / C1** | Regulated +5 V DC | Buck 1 output | Servo VCC, motor-driver VCC/nSTBY logic |
| **Buck 2 output** | 5 V DC, 5 A rated | Buck 2 | Raspberry Pi 5 and connected peripherals (see §7.9 for load budget) |
| **3.3 V** | Regulated +3.3 V DC | Raspberry Pi 5 pin 1 | OLED VCC, push button supply and IMU VCC |
| **GND** | 0 V reference | Battery/system ground | Common ground across the system |

## 7.8 Battery availability

The team had three batteries and could charge only one at a time.

This reduced the number of consecutive tests that could be performed before charging became necessary.

## 7.9 Power Budget

To check whether each regulated rail has adequate headroom, the expected current draw of each connected load was estimated from manufacturer and publicly documented specifications, since a full bench current-logging setup was not available during this development phase.

**+5 V servo/logic rail (Buck 1, rated 5 V / 3 A):**

- DS3225 servo, idle/holding: approximately 4–5 mA.
- DS3225 servo, stall: published manufacturer specifications for this servo range from approximately 1.9 A to 3.5 A depending on the source, so this figure carries meaningful uncertainty.
- Motor-driver logic (VCC/nSTBY) and push-button: combined, well under 10 mA.

Even using the lower end of the published servo stall-current range, this rail has limited headroom against its 3 A rating. Using the upper end of the range, the servo alone could approach or exceed the rail's rated capacity during a hard steering stall. This is flagged as an open risk rather than a resolved one: the team's next step is to bench-measure the actual stall current of the installed servo directly, since manufacturer figures for this part disagree significantly across sources.

**+5 V Raspberry Pi rail (Buck 2, rated 5 V / 5 A):**

- Raspberry Pi 5, idle: approximately 0.6 A.
- Raspberry Pi 5, typical vision-processing load: approximately 1.3 A.
- Raspberry Pi 5, worst-case peak: approximately 2.3 A.
- Two Raspberry Pi Camera Module 3 units, active: approximately 0.25 A each, 0.5 A combined.

Summed worst-case draw on this rail is approximately 2.8 A, leaving roughly 2.2 A of headroom against the 5 A rating.

The drive motor is not powered through either regulated rail; it draws directly from the raw battery line through the motor driver, and its current behaviour is already characterised in Section 6.3.

The push-button, LEDs and MPU6050 IMU are powered from the Raspberry Pi's own regulated 3.3 V rail rather than from either buck converter. Their combined draw is a few milliamps at most and is negligible relative to the capacity of that rail, so it is not included as a separate line in the budget above.

| | |
|---|---|
| **Problem** | Whether the regulated rails have sufficient current headroom for their connected loads. |
| **Constraint** | Bench current-logging equipment was not available; loads had to be estimated from published specifications rather than measured directly. |
| **Decision** | Datasheet-derived current estimates were used to build a worst-case budget for both regulated rails. |
| **Result / next step** | The Raspberry Pi rail has comfortable headroom. The servo/logic rail has limited to marginal headroom depending on the servo's true stall current, which is flagged as an item to verify by direct measurement. |

# 8 Camera System and Sensor Placement

## 8.1 Why cameras instead of colour sensors?

The team selected cameras because the robot needed to reason about multiple visual features at once.

The required visual tasks include:

- black track boundaries,
- blue/orange direction markers,
- red obstacles,
- green obstacles,
- magenta parking structures.

A camera provides spatial information as well as colour information. This makes it possible to estimate where an object is in the image and use that location as a steering target.

Figure 22: Final two-camera arrangement. F and B identify the front and back of the robot.

## 8.2 Camera hardware

Both cameras use ribbon-cable connections to the Raspberry Pi camera interface.

The measured heights from the ground to the centre of the lens are:

Table 9: Camera placement measurements

| Camera | Lens-centre height |
|---|---|
| Main perception camera | 25.0 cm |
| Parking camera | 23.5 cm |

Table 10: Camera resolution and frame rate

| Camera | Resolution | Frame rate |
|---|---|---|
| Front camera (Open & Obstacle) | 1480 × 520 | 60 FPS (target) |
| Back / parking camera | 640 × 480 | 60 FPS |

## 8.3 Camera interface

Both Raspberry Pi Camera Module 3 cameras connect to the Raspberry Pi using the ribbon-camera interface. This provides the camera data connection while keeping the cameras physically separated for their forward and rearward roles.

## 8.4 Two-camera architecture

The final robot uses two cameras.

- The forward-looking camera is used primarily for line and obstacle perception.
- The second camera provides rearward information needed for parking.

The two cameras are mounted toward the rear of the robot while facing opposite useful directions. This arrangement allows the robot to retain a forward field of view while also obtaining information about the area behind it.

Figure 23: Forward-looking camera and adjustable mount.

## 8.5 Camera placement experiments

Camera placement was one of the largest perception problems.

The team experimented with several arrangements. One high-position configuration provided a useful view of the track but made obstacle detection less effective.

The camera was therefore lowered and its angle adjusted.

The design objective became:

$$\text{Track visibility} + \text{Obstacle visibility} + \text{Stable mounting}.$$

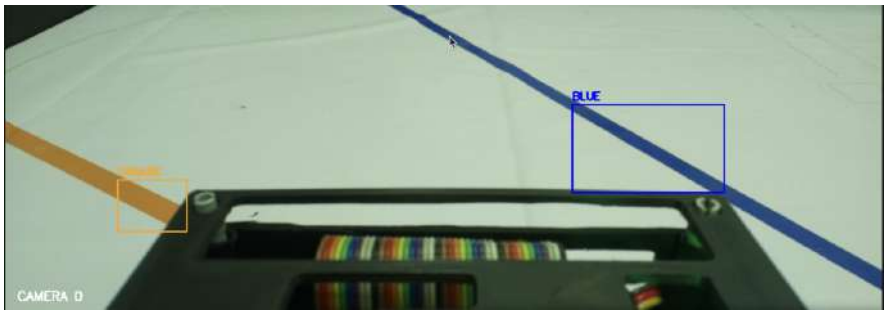


Figure 24: Representative forward camera view used for track perception.

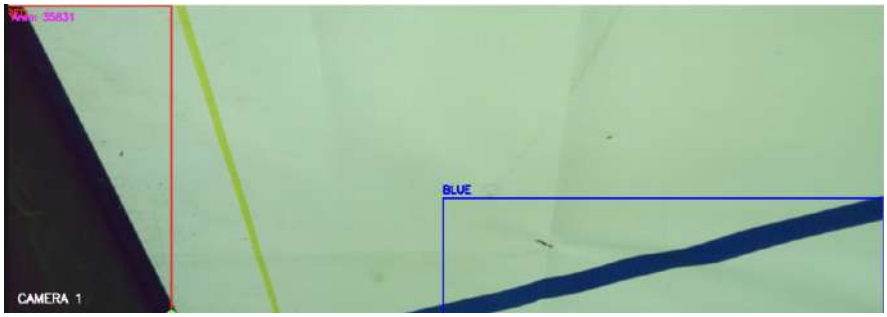


Figure 25: Rear-camera view showing the space beyond the parking area.

The final arrangement was chosen experimentally rather than from a single calculated camera angle.

### 8.6 Why an adjustable mount mattered

The relationship between camera angle and image coordinates is important to the control system.

A camera that is too high can make obstacles appear smaller and farther away in the image. A camera that is too low can reduce the amount of track visible ahead.

Therefore, the mount was designed to allow the team to change the view and observe the resulting effect directly.

## 9 Computer Vision Pipeline

### 9.1 Implemented processing sequence

The current vision implementation is based on direct colour segmentation and contour analysis in OpenCV. The Open and Obstacle Challenge programs run different processing sequences.

**Open Challenge.** Detections are accepted at a fixed confidence once they pass a minimum area threshold, and the full frame is used with no ROI crop:

$$\text{Camera frame} \rightarrow \text{HSV conversion} \rightarrow \text{colour mask} \rightarrow \text{opening/closing} \rightarrow \text{contours} \rightarrow \text{target point}$$

Both challenges currently run the front camera at 1480 x 520 pixels, targeting 60 FPS. This resolution was chosen to prioritise frame rate over image detail, since a smaller frame processes faster and reduces the chance of missing a fast-moving detection target.

**Obstacle Challenge.** Detections are additionally weighted using the confidence formula in Section 9.2, combining HSV coverage, an LAB colour-consistency check, and geometric rectangularity. The lower 120 pixels of the frame are excluded from detection, since this region can contain the robot's own body:

$$\begin{aligned} &\text{Camera frame} \rightarrow \text{ROI crop} \rightarrow \text{HSV conversion} \rightarrow \text{colour mask} \\ &\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \downarrow \\ &\text{target point} \leftarrow \text{confidence filter} \leftarrow \text{contours} \leftarrow \text{opening/closing} \end{aligned}$$

Using a higher resolution specifically for the Obstacle Challenge, where colour and position accuracy at range matters more than raw frame rate, is a planned next step rather than the current implementation.

### 9.2 Colour confidence scoring

A candidate contour must satisfy a minimum area and a combined confidence threshold. The implementation combines three terms:

$$C = 0.65C_{HSV} + 0.20C_{LAB} + 0.15C_{geometry}.$$

HSV coverage provides the main colour evidence, a LAB comparison provides an additional colour-consistency check, and rectangularity provides a geometric score. Detections below the configured confidence threshold are rejected.

### 9.3 Morphological processing and contours

Each colour mask is cleaned using morphological opening followed by closing with a small elliptical kernel. The software then extracts external contours and rejects candidates below the minimum area. For each accepted detection it records the bounding rectangle, centre point, area and confidence.

Figure 26: Black segmentation mask used to isolate track boundaries.

### 9.4 Colour-channel correction

During camera testing, the team observed that colours could appear inverted or assigned to the wrong labels when the image channel interpretation did not match the camera output. This was corrected in the vision implementation. The issue was identified during normal-colour testing rather than through a synthetic test image, making it important to verify the complete camera-to-detection pipeline on the physical robot.

### 9.5 Measured processing performance

Three captured test overlays recorded the following values:

Table 11: Observed camera and processing performance in obstacle challenge

| Metric | Test 1 | Test 2 | Test 3 | Mean |
|---|---|---|---|---|
| Camera FPS | 11.24 | 10.88 | 11.55 | 11.22 |
| Loop FPS | 11.32 | 10.88 | 11.56 | 11.25 |
| Capture latency (ms) | 1.90 | 2.16 | 1.80 | 1.95 |
| Loop latency (ms) | 79.08 | 90.87 | 82.73 | 84.23 |

These values are observed samples from the current on-robot test overlays, not a claim that the system always operates at the same rate. The configured camera target remains 60 FPS, while the observed end-to-end processing loop in these tests was approximately 11 FPS.

## 10 Colour Detection and Object Interpretation

### 10.1 Colour roles

The software uses different colours for different tasks.

Table 12: Colour interpretation in the software

| Colour | Role |
|---|---|
| Black | Track boundaries / line following |
| Blue | Direction marker / lap-counting marker |
| Orange | Direction marker / lap-counting marker |
| Green | Obstacle detection and steering response |
| Red | Obstacle detection and steering response |
| Magenta / purple | Parking structures |

### 10.2 Blue and orange marker logic

The playfield contains four blue and four orange marker lines.

The program does not need to track both marker colours after the direction is known.

The first valid marker determines the direction:

$$\text{Blue} \rightarrow \text{Anticlockwise}$$

and:

$$\text{Orange} \rightarrow \text{Clockwise}.$$

The program then counts only the marker corresponding to that direction.

### 10.3 Why only one marker colour is counted

Detecting both marker colours continuously would add unnecessary decision logic because only one marker type is relevant after the direction has been established.

The simpler strategy is:

1. detect the first marker,
2. determine direction,
3. select the appropriate marker colour,
4. count only that colour.

This reduces the possibility of the opposite marker interfering with lap counting.

## 11 Open Challenge Software Architecture

### 11.1 Program structure

The Open Challenge program can be understood as five functional layers:

1. hardware initialisation,
2. camera acquisition and preprocessing,
3. track/marker perception,
4. direction and lap management,

5. steering control and termination.

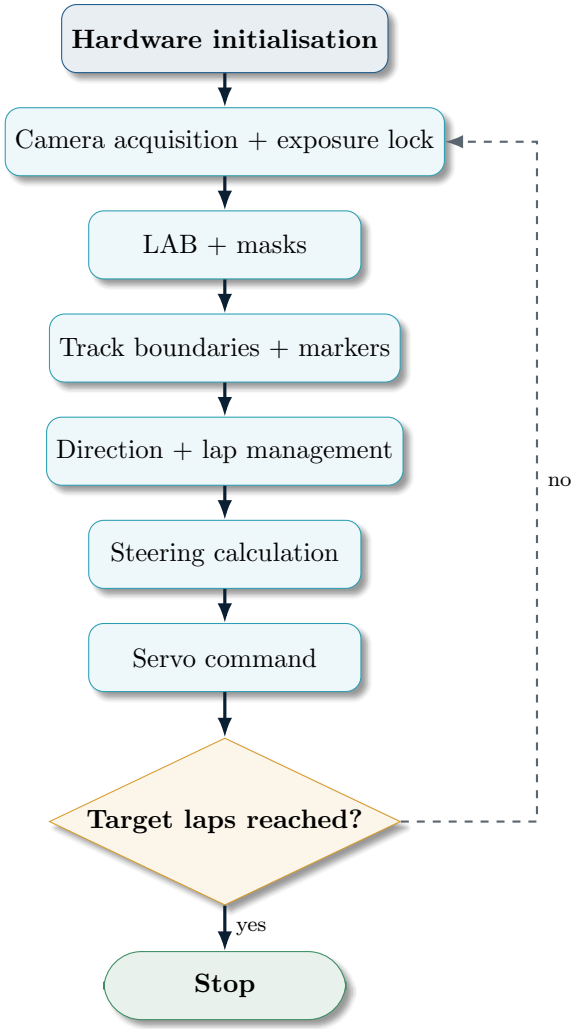


Figure 27: Open Challenge software architecture.

### 11.2 Camera exposure locking

At startup the camera is allowed to auto-adjust exposure and white balance.

The program then records the resulting exposure and analogue gain and disables automatic adjustment.

This creates a more stable imaging condition during the run and reduces the possibility that the segmentation thresholds change because the camera continuously changes its exposure.

### 11.3 Region of interest

The Open Challenge program removes a lower central region of the image from black-marker processing because this region can contain the robot itself.

The idea is to prevent the robot body from becoming a false black contour.

This is an example of using knowledge of the physical robot to improve perception.

## 12 Steering Control

### 12.1 Proportional control

The Open Challenge uses a proportional controller.

The general form is:

$$u = K_P e$$

and the steering command is:

$$\theta = \theta_c + K_P e.$$

For the Open Challenge implementation:

$$K_P = 0.013.$$

The value was experimentally tuned.

### 12.2 Two-wall case

When both useful boundaries are visible, the program calculates:

$$d_L = x_L$$

and:

$$d_R = W - x_R$$

where $W$ is the image width.

The steering error is:

$$e = d_L - d_R.$$

The sign and magnitude of this error determine the direction and amount of steering correction.

### 12.3 One-wall cases

Both walls are not always visible.

When only the left boundary is visible, the program estimates steering from the left target.

When only the right boundary is visible, it does the equivalent using the right target.

This makes the controller less dependent on having a complete view of both boundaries.

### 12.4 No-wall case

When no useful boundary is detected, the Open Challenge program returns to a centre steering command.

This is intentionally conservative: it avoids generating an arbitrary large correction from missing data.

### 12.5 Steering saturation

The command is clamped between the calibrated steering limits:

$$35^\circ \leq \theta \leq 115^\circ.$$

This protects the mechanical system from software commands beyond the tested servo range.

## 13 Marker Counting and Lap Management

### 13.1 The duplicate-counting problem

A line or coloured marker does not exist for only one camera frame. As the robot moves through it, the marker can remain visible across many consecutive frames.

A naive program could therefore produce:

$$1 \text{ physical crossing} \rightarrow 10 \text{ detected frames} \rightarrow 10 \text{ counted crossings.}$$

This would cause the robot to stop too early.

### 13.2 Rising-edge detection

The Open Challenge implementation stores whether the selected marker was visible in the previous frame.

A crossing is counted only when:

$$\text{marker}_{current} = 1$$

and:

$$\text{marker}_{previous} = 0.$$

This is a rising-edge condition.

### 13.3 Cooldown

A cooldown is also applied.

The Open Challenge code uses:

$$T_{\text{cooldown}} = 1.0 \text{ s.}$$

The purpose is to stop the same physical crossing from being counted again immediately.

The value was experimentally adjusted during testing.

### 13.4 Lap count

The program is configured for:

$$3 \text{ laps} \times 4 \text{ relevant crossings per lap} = 12$$

counted crossings.

Once the count reaches the configured target, the robot centres the steering and stops.

## 14 Obstacle Challenge Architecture

### 14.1 Purpose

The Obstacle Challenge requires the robot to do more than follow a boundary. It must recognise coloured obstacles and alter its path.

The obstacle program therefore gives obstacle detections higher priority than ordinary wall-following targets.

### 14.2 Detection hierarchy

The implemented steering logic can be represented as:

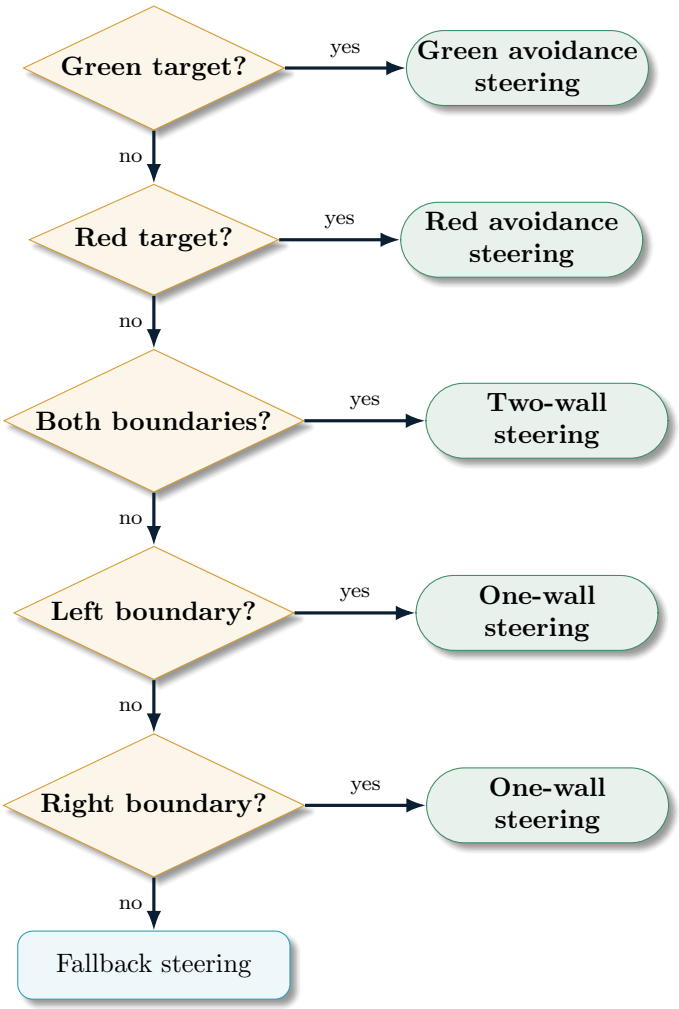


Figure 28: High-level steering priority in the obstacle program.

The important idea is that an obstacle target should influence steering before ordinary wall-following targets when the obstacle is relevant.

## 14.3 Green obstacle detection

The program creates a green mask, performs morphological processing, extracts contours and chooses the largest contour.

A target point is then created at the bottom of the detected obstacle's bounding rectangle.

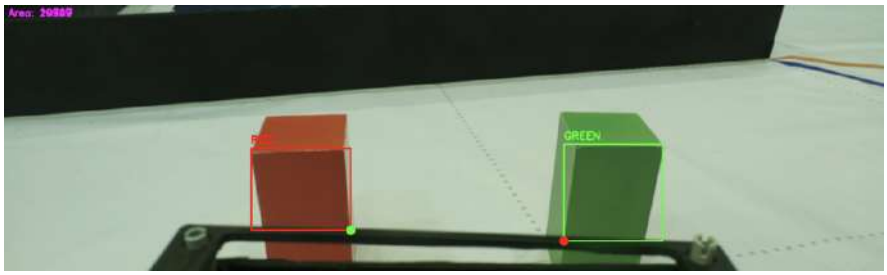


Figure 29: Obstacle detection output showing the colour-segmentation and contour stage.

## 14.4 Red obstacle detection

The red pipeline follows a similar process.

The largest qualifying contour is selected and its bounding rectangle is used to derive a target point.

## 14.5 Direction-dependent steering

The exact steering offset depends on the direction and the detected image position.

This is necessary because the same image coordinate does not represent the same avoidance manoeuvre when the robot travels in the opposite direction.

The team therefore did not use one universal obstacle-steering formula.

### 14.6 Program flow: laps, obstacle avoidance and parking approach

The obstacle program follows a single continuous sequence rather than switching between entirely separate modes for laps and parking.

The robot completes three laps by steering around each detected obstacle as it appears, using the same green/red target-point logic described above. Laps are not counted by corner markers alone; instead, the program tracks how many times the purple/magenta parking marker has been detected by the rear camera, since this marker is passed once per lap.

On the third detection of the parking marker, the robot does not stop immediately. It continues driving for one additional section past this point, avoiding any remaining obstacles, then performs a U-turn and drives back to enter the parking area. This ensures the robot is never mid-avoidance when it commits to parking.

Once this condition is satisfied, the parking approach becomes direction-dependent:

- If the direction is clockwise, the robot follows the left-side wall until only the front wall is visible in the camera frame.
- It then performs two successive 90° turns to return to the same side of the track it came from.
- The robot then drives forward along the right-side wall until the rear camera detects the purple parking marker.
- From this point, a series of guided turns (detailed in the source code) brings the robot into the parking area.

This direction-dependent approach was necessary because a single fixed sequence of turns does not produce the correct final heading for both possible track directions; the wall-following side and turn sequence must be mirrored depending on which direction the round is run in.

## 15 Parking and IMU Control

### 15.1 Why timed turns were insufficient

An early approach considered the duration of a turn as a proxy for the amount of rotation.

The problem is that a timed turn does not directly measure orientation.

The actual angle achieved can vary because of:

- battery condition,
- motor speed,
- friction,
- wheel contact,
- steering geometry,
- acceleration.

The team therefore moved toward IMU-based orientation feedback.

## 15.2 MPU6050 and implemented calibration

The robot uses an MPU6050. The heading implementation performs a stationary Z-axis gyroscope calibration at startup. After an initial settling period, it collects 1500 gyroscope samples while the sensor is kept still, averages those samples to estimate the Z-axis bias, and subtracts that offset from subsequent readings. The corrected angular velocity is then integrated over elapsed time and wrapped to a 0–360 degree heading.

The parking implementation reads this heading during the manoeuvre and uses it to determine whether the desired orientation has been reached. This is an implemented bias-calibration procedure rather than an unresolved calibration item; the code currently uses gyroscope-based heading feedback and does not claim a separate absolute heading reference.

The gyroscope is read over I2C at its default ±250°/s sensitivity range (131 LSB per °/s), and the full calibration sequence, including the initial settling period, takes approximately three seconds.

The MPU6050 datasheet specifies zero-rate output variation of up to ±20°/s across its full −40°C to +85°C rated temperature range; this figure is not directly applicable to a short indoor run at roughly constant temperature, where the residual bias after calibration is expected to be substantially smaller. However, because the heading calculation integrates angular velocity with no magnetometer or other absolute reference to correct it, even a small residual bias accumulates linearly with time: a residual of just 0.1°/s, for example, would produce approximately 18° of accumulated heading error over a full 3-minute round. This illustrates why gyro-only heading is treated as reliable over the short duration of a single manoeuvre (such as a parking turn) but is not assumed to hold with the same accuracy across an entire round

## 15.3 Parking state machine

The parking implementation contains explicit state concepts rather than one continuous steering expression.

A simplified representation is:

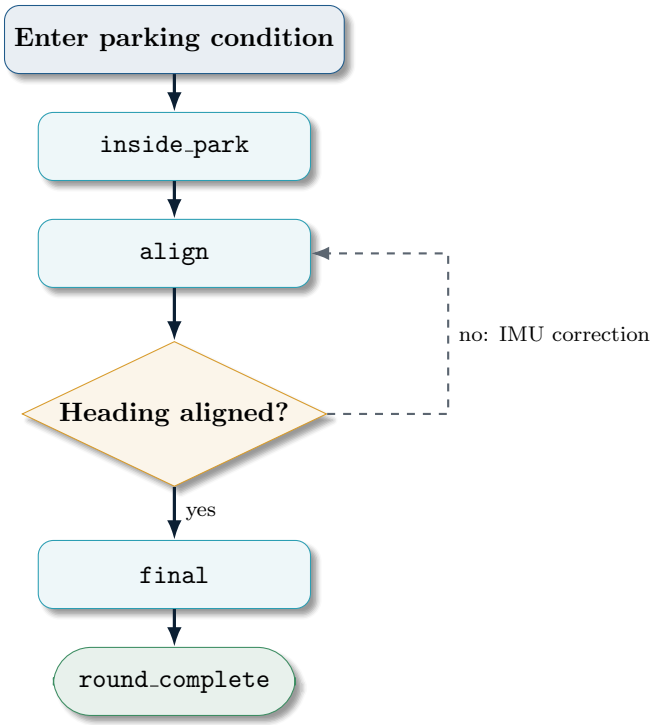


Figure 30: Simplified representation of the parking state logic.

## 15.4 Parking visual detection

The rear camera is used because the robot needs information about the parking area behind it.

The parking structures are identified using colour segmentation and contour filtering.

### 15.5 Current parking status

Parking is now working reliably in the team's current tested configuration. The earlier limitation involving the final return into the parking area was addressed through the rear-camera, state-based and IMU-heading approach described above.

Further testing can still be used to quantify performance across a larger range of obstacle placements and track conditions, but the parking sequence is no longer listed as an unresolved implementation problem.

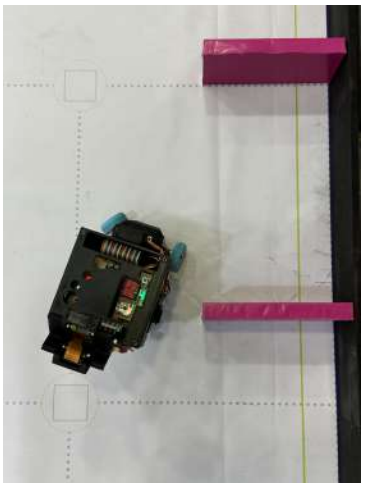

Figure 31: Parking test area and the configuration used during parking development.

## 16 Testing Methodology

### 16.1 Why testing was iterative

The robot could not be designed completely on paper because several important behaviours depend on the interaction between:

- physical geometry,
- camera perspective,
- lighting,
- wheel motion,
- servo response,
- battery state,
- software thresholds.

Consequently, the team used repeated physical testing and adjusted parameters experimentally.

### 16.2 Parameter tuning

Several important values were tuned experimentally:

- camera colour thresholds,
- contour-area thresholds,
- proportional gain $K_P$,
- servo centre,
- servo limits,
- line cooldown,

- camera angle,
- operating speed.

The colour thresholds were initially based approximately on online reference values and then adjusted using the actual camera and track.

This is an important distinction:

> **Reference values provided the starting point; physical testing provided the final values.**

### 16.3 Lighting

Lighting changes can alter camera segmentation.

The software therefore locks exposure and analogue gain after an initial automatic adjustment.

The robot also uses an LED close to the relevant viewing area, reducing the effect of changing ambient conditions.

The system is not claimed to be completely illumination-independent.

## 17 Open Challenge Results

### 17.1 Recorded runs

The following six complete Open Challenge runs were recorded in the anticlockwise direction.

Table 13: Six recorded Open Challenge runs

| Run | Completion time | Observation |
|---|---|---|
| 1 | 32 s | Initial recorded run |
| 2 | 30 s | Improvement |
| 3 | 28 s | Improvement |
| 4 | 28 s | Stable |
| 5 | 28 s | Stable |
| 6 | 28 s | Stable |

**28 s**
Best recorded complete Open Challenge time

**28 s**
Mode reached in each of the final four recorded runs

The sequence indicates that performance stabilised at 28 seconds in the later recorded runs.

It would be incorrect to claim from this small dataset that the robot has a statistically established 28-second performance. The useful conclusion is that repeated runs under the tested conditions reached the same completion time.

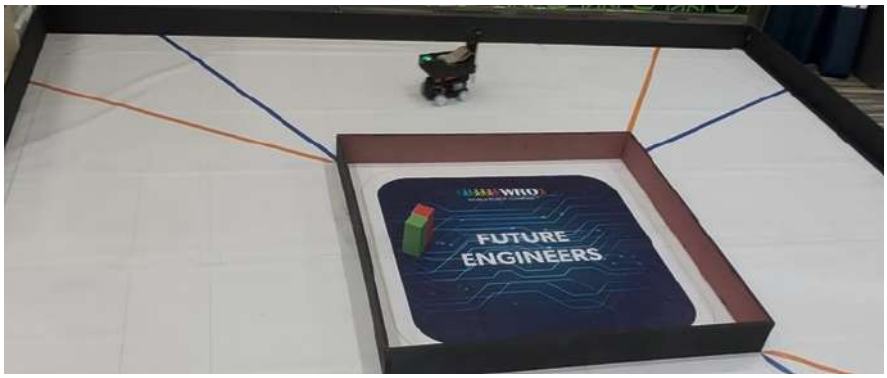


Figure 32: Open Challenge testing environment.

# 18 Obstacle Challenge Results

## 18.1 Recorded complete runs

With the current parking sequence working reliably, six complete robot runs were recorded with the following completion times:

Table 14: Six recorded complete runs with the current obstacle and parking system

| Run | Completion time | Observation |
|---|---|---|
| 1 | 1 min 10 s | Initial recorded run |
| 2 | 1 min 09 s | Improvement |
| 3 | 1 min 11 s | Near the initial result |
| 4 | 1 min 20 s | Slower placement/test configuration |
| 5 | 1 min 15 s | Recovery after slower run |
| 6 | 1 min 13 s | Improved from run 5 |

The completion times range from 1 min 09 s to 1 min 20 s. They should not be treated as directly identical trials because obstacle placement was slightly different between runs, which changed the path and time required. The useful engineering improvement is that the current system can complete the full sequence, including parking, rather than treating parking as an unresolved final-stage failure.

**1 min 09 s**
Best recorded complete run time


Figure 33: Obstacle Challenge testing environment.

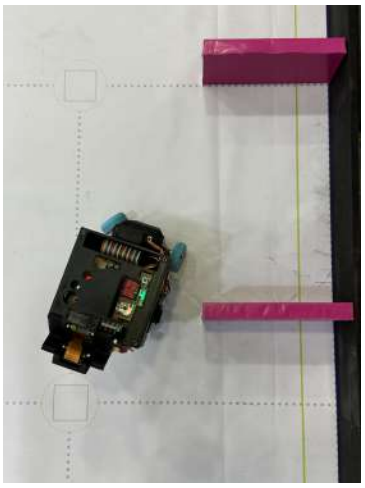

Figure 34: Parking test setup.

### 18.2 Why the run time is not the only metric

A fast run is useful only if the robot remains on the track and completes the required task.

The project therefore prioritises:

$$\text{accuracy} > \text{repeatability} > \text{raw maximum speed}.$$

This explains why the operating speed was intentionally kept lower during some stages of development.

## 19 Failure Analysis

### 19.1 Failure: motor stalling

**Observed problem:** The original motor could stall.

**Risk:** Increased current demand could exceed the safe operating capability of the motor driver.

**Response:** Replace the motor with the JGB37-520 600 RPM unit.

**Trade-off:** Lower maximum speed.

**Engineering lesson:** Electrical reliability is part of mechanical performance.

### 19.2 Failure: camera mount wobble

**Observed problem:** The original LEGO camera mount was wobbly.

**Risk:** Camera movement changes image geometry and therefore changes the relationship between detected objects and steering.

**Response:** CAD-designed camera mount.

**Result:** More rigid and adjustable camera positioning.

### 19.3 Failure: poor obstacle visibility

**Observed problem:** A high camera position did not provide the desired obstacle detection.

**Response:** Lower the camera and adjust its angle.

**Result:** Better balance between track and obstacle visibility.

### 19.4 Failure: duplicate line detection

**Observed problem:** One physical marker could be visible across multiple frames.

**Response:** Rising-edge detection and cooldown.

**Result:** Reduced repeated line counts.

### 19.5 Failure: excessive speed

**Observed problem:** While using the higher-power Johnson motor, at very high speed the robot could lift its front wheels and steering became less forgiving.

**Response:** Reduce operating speed and tune steering.

**Result:** Better control, at the cost of maximum speed.

### 19.6 Failure: wall collisions

**Observed problem:** The robot can still collide with walls, particularly in obstacle-oriented situations.

**Response so far:** Improve target selection, direction-specific steering and perception.

**Status:** Not completely solved.

## 20 Engineering Trade-offs

This section records decisions where improving one property required accepting a cost elsewhere.

Table 15: Major engineering trade-offs

| Decision | Benefit | Cost | Final judgement |
|---|---|---|---|
| 4WD | Power delivered to all four wheels | More mechanical complexity | Accepted because all-wheel power was a design priority |
| Mechanical differential | Rule-compatible mechanical solution | More drivetrain complexity | Accepted; LEGO differential was effective |
| Johnson 1000 RPM → JGB37 600 RPM | Lower stall current | Lower rotational speed | Accepted for reliability |
| LEGO → CAD chassis | Greater rigidity and custom mounting | CAD design/manufacturing effort | Accepted |
| LEGO camera mount → CAD mount | More stable camera geometry | Additional design/manufacturing | Accepted |
| One camera → two cameras | Rearward parking perception | More hardware/software complexity | Accepted because parking required rear information |
| Higher operating speed | Potentially lower lap time | More difficult steering and stability | Reduced in favour of accuracy |
| Timed parking turns → IMU feedback | Direct orientation information | Additional sensor and software complexity | Accepted; parking still being tuned |
| Both-wall steering → one-wall fallback | Continued operation when one wall disappears | Less complete geometric information | Accepted because both walls are not always visible |

## 21 Systems Thinking

### 21.1 Mechanical-electrical interaction

Changing the motor changed more than the drive speed.

It affected:

$$\text{motor} \rightarrow \text{current demand} \rightarrow \text{driver risk} \rightarrow \text{power architecture} \rightarrow \text{available operating speed}.$$

Therefore, the motor decision could not be evaluated solely using RPM.

### 21.2 Mechanical-vision interaction

The camera mount is physically part of the chassis but functionally part of the perception system.

A wobbling camera changes the image. A changed image changes the detected target. A changed target changes steering.

Thus:

$$\text{mount stability} \rightarrow \text{image stability} \rightarrow \text{target stability} \rightarrow \text{steering stability}.$$

This is why the camera mount became a CAD component.

### 21.3 Software-mechanical interaction

The steering limits in software are constrained by the physical servo and steering mechanism.

The software cannot be designed independently of:

- servo centre,
- mechanical steering range,
- wheel geometry,
- chassis geometry.

The calibrated values therefore form part of the mechanical/software interface.

### 21.4 Perception-control interaction

A perception algorithm can be accurate in isolation but still produce poor robot behaviour if its output is too late, too noisy or mapped to the wrong steering target.

This became particularly visible in obstacle avoidance.

The remaining obstacle failures therefore indicate a systems problem, not simply a “bad colour mask”.

## 22 Engineering Risk Register

Table 16: Current risk register

| Risk | Mitigation | Current status |
|---|---|---|
| Motor stall | Lower-risk motor selection | Reduced |
| Motor-driver overload | Motor replacement and separate power design | Reduced |
| Power instability | Separate regulated branches | Improved |
| Camera wobble | CAD camera mount | Improved |
| Lighting variation | Locked exposure/gain and nearby LED | Reduced, not eliminated |
| Duplicate line count | Rising edge + cooldown | Reduced |
| High-speed instability | Lower operating speed | Controlled by tuning |
| Wall collision | Improved target logic | Unresolved in some cases |
| Battery availability | Three batteries / charging management | Resource constraint |
| Battery charging failure | Careful battery handling | One observed incident; cause unconfirmed |

# 23 Reproducibility and GitHub

## 23.1 Repository

Our project repository is:

https://github.com/darsh-makadia/Team-Current-WRO-FE-2026

Our repository contains the project's:

- source code,
- CAD models,
- schematics,
- photographs,
- supporting documentation,
- testing material.

## 23.2 Hardware reproducibility

A person attempting to reproduce the robot should have access to:

- Raspberry Pi 5 4 GB,
- two Raspberry Pi Camera Module 3 cameras,
- JGB37-520 12 V 600 RPM motor,
- servo motor,
- MPU6050,
- TB6612FNG motor driver,
- 3S 2200 mAh LiPo,
- buck converter(s),
- PLA,

- required LEGO drivetrain components.

### 23.3 Software reproducibility

The software depends on the Raspberry Pi camera environment and libraries used in the source code, including OpenCV, NumPy, RPi.GPIO and Picamera2.

The repository should therefore be treated as the authoritative source for the exact implementation.

### 23.4 Configuration values

Important control values are explicitly present in the code.

Examples from the Open Challenge program include:

```
LAPS_TO_COMPLETE = 3
LINES_PER_LAP = 4
LINE_COOLDOWN = 1
KP = 0.013

CENTER = 75
LEFT = 35
RIGHT = 115
```

These values are not presented as universal constants. They are the experimentally tuned values used by this version of the robot.

## 24 Repository Content Map

The following is a content map for the material that is organised as follows.

Table 17: Recommended repository organisation

| Directory / file | Purpose |
|---|---|
| `README.md` | Project overview, hardware and setup |
| `code/` | Challenge software |
| `cad/` | Final and relevant CAD models |
| `schematics/` | Electrical diagrams |
| `photos/` | Robot and development photographs |
| `testing/` | Test results and supporting evidence |
| `documentation/` | Engineering journal and supporting documents |

The repository structure shown here follows the organisation we use for the project.

## 25 Final Design Summary

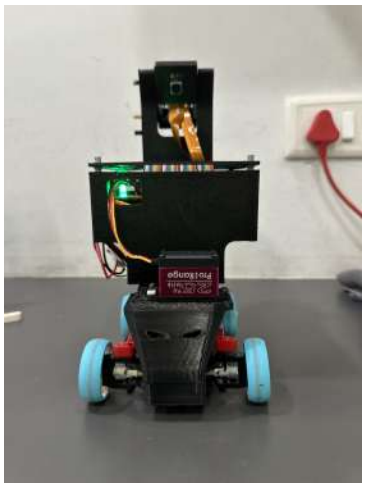

Figure 35: The Dark Knight — final design.

Figure 36: Team Current.

Table 18: Final system architecture

| Subsystem | Final implementation |
|---|---|
| Mechanical frame | CAD/PLA custom chassis |
| Drivetrain | Four-wheel drive |
| Differential | Mechanical LEGO differential |
| Steering | Servo steering |
| Drive motor | JGB37-520 12 V 600 RPM |
| Computer | Raspberry Pi 5 4 GB |
| Vision | Two cameras + OpenCV |
| Image representation | LAB + HSV |
| Track detection | Black masks + contours |
| Marker detection | Blue/orange masks + contours |
| Obstacle detection | Green/red masks + contours |
| Parking detection | Magenta mask + contours |
| Orientation | MPU6050 |
| Power | 3S LiPo + separate regulated branches |
| Structural printing | PLA / Anycubic Cobra 2 Neo |

Figure 37: Rear view of the final robot and parking-oriented camera architecture.

# 26 Current Performance and Limitations

The team considers the project an evolving engineering system rather than a finished product.

## 26.1 Strongest current subsystem

The strongest demonstrated is the Open Challenge followed by Obstacle Challenge with parking.

The six recorded runs produced for open challenge:

$$32,\ 30,\ 28,\ 28,\ 28,\ 28 \text{ seconds.}$$

The repeated 28-second runs provide evidence of repeatability under the tested conditions.

The six recorded runs produced for the Obstacle Challenge:

$$1 \text{ min } 10,\ 1 \text{ min } 09,\ 1 \text{ min } 11,\ 1 \text{ min } 20,\ 1 \text{ min } 15,\ 1 \text{ min } 13 \text{ seconds.}$$

The completion times ranged from 1 minute 09 seconds to 1 minute 20 seconds. Because obstacle placement varied slightly between runs, these should not be treated as directly identical trials; however, the robot successfully completed the full sequence, including obstacle navigation and parking, under the tested conditions.

# 27 Conclusion

The Dark Knight developed from a LEGO-heavy prototype into a hybrid autonomous vehicle through repeated engineering decisions.

The most important decisions were not isolated component choices. They were responses to interactions between constraints.

The differential problem led to a mechanical LEGO solution. The limitations of the LEGO chassis led to CAD structural components. The motor stalling problem led to a lower-risk motor. The power problem led to separate branches. Camera placement experiments led to an adjustable mount and eventually a two-camera architecture. Computer-vision limitations led to LAB processing, morphology, contour filtering and experimentally tuned thresholds. Line-counting problems led to rising-edge and cooldown logic. Earlier parking limitations led to IMU heading feedback and an explicit parking state sequence, which now works reliably in the team's current tested configuration.

The project therefore represents a continuous loop:

Problem → Constraint → Design decision → Implementation → Test → Observation → Next iteration

The strongest evidence of this process is not the final robot alone. It is the chain of decisions that produced it.

The engineering philosophy behind The Dark Knight remains:

**Use the materials we have intelligently,**
**understand their limitations,**
**and optimise the complete system rather than one component.**

# A Complete Component Inventory

## A.1 LEGO mechanical components

- 2X4 L beam
- Bush
- Half bush
- 2L axle connector
- 28-tooth differential
- 24-tooth gear
- 20-tooth gear
- 12-tooth bevel gear
- 9 beam
- 5L axle
- 6L axle
- 4L axle with stop
- Smooth pin
- Friction pin
- Universal joint
- 3L friction pin

## A.2 Purpose of the LEGO mechanical elements

The LEGO elements were not treated as generic construction pieces. Each retained component served a specific mechanical purpose in the drivetrain, steering or structural integration.

| Component | Purpose in the robot |
|---|---|
| 2X4 L beam | Structural support and mounting geometry for the LEGO mechanical assembly. |
| Bush | Axial spacing and retention of rotating LEGO axles. |
| Half bush | Fine axial spacing where a full bush would provide too much clearance. |
| 2L axle connector | Joins axle sections and transfers rotation through the drivetrain. |
| 28-tooth differential | Provides the mechanical differential function so the driven wheels can accommodate different rotational speeds while turning. |
| 24-tooth gear | Transfers torque between drivetrain shafts and establishes the required gear relationship. |
| 20-tooth gear | Intermediate drivetrain gear used to transmit motion and set the final mechanical ratio. |
| 12-tooth bevel gear | Transfers rotational motion through a change in shaft direction within the compact drivetrain. |
| 9 beam | Rigid LEGO frame member used to locate and support drivetrain components. |

| | |
|---|---|
| 5L axle | Transfers rotational motion between gears and drivetrain components. |
| 6L axle | Longer shaft for spanning the required distance between supported drivetrain elements. |
| 4L axle with stop | Short axle used where controlled axial positioning is required. |
| Smooth pin | Low-friction connection used where a movable LEGO joint is required. |
| Friction pin | Rigid structural connection between LEGO beams/components. |
| Universal joint | Transfers rotation between shafts that are not perfectly collinear. |
| 3L friction pin | Compact rigid connector for attaching small drivetrain/structural elements. |

The important design decision was to keep LEGO for compact mechanical features such as the differential and gear train, while using CAD/PLA for the large rigid structures. This avoided unnecessarily redesigning accurate LEGO gearing while gaining the rigidity and custom geometry required for the chassis, camera mount and electronics enclosure.

### A.3 Electronic and electrical components

- Raspberry Pi 5, 4 GB
- Raspberry Pi Camera Module 3 ×2
- Servo motor
- JGB37-520 DC 12 V 600 RPM motor
- Johnson gear motor, 1000 RPM — earlier prototype
- TB6612FNG motor driver
- MPU6050 IMU
- OLED display
- RGB LED
- Programmable LED
- 3S LiPo battery, 12 V nominal, 2200 mAh
- DC–DC buck converter
- USB buck converter

### A.4 Manufacturing

- PLA filament
- Anycubic Cobra 2 Neo 3D printer

## B Software Parameter Reference

Table 20: Selected Open Challenge parameters (tuned for higher FPS and corner-detection reliability at speed)

| Parameter | Value |
|---|---|
| Image width | 1480 px |
| Image height | 520 px |
| $K_P$ | 0.013 |
| Line cooldown | 1.0 s |
| Laps | 3 |
| Relevant crossings per lap | 4 |
| Total counted crossings | 12 |
| Servo centre | 75° |
| Servo minimum | 35° |
| Servo maximum | 115° |
| Initial motor PWM | 40 |

Table 21: Selected Obstacle Challenge parameters

| Parameter | Value |
|---|---|
| Image width | 1480 px |
| Image height | 520 px |

The Obstacle Challenge uses the higher resolution because obstacle colour and position must be resolved accurately at range, which matters more here than the frame rate gains that motivate the Open Challenge's lower resolution (Section 9.1).

## B.1 Vision parameters

The exact colour thresholds should be reproduced from the final source code in the repository. The thresholds were experimentally tuned from initial approximate values.

```
BLACK_LOWER = [0, 118, 118]
BLACK_UPPER = [75, 138, 138]

BLUE_LOWER = [0, 120, 80]
BLUE_UPPER = [255, 150, 120]

ORANGE_LOWER = [140, 120, 145]
ORANGE_UPPER = [210, 155, 210]
```

The Obstacle Challenge adds green and red thresholds, which are likewise defined in the challenge source code.